

Notice des essais sur le GPS

Ce document contient une description des essais concernant le GPS dont sont équipés les robots scout mini et bunker mini + devkit ROS2.

Ces essais ont été effectués sur le robot scout mini 07.

But des essais

Le but de ces essais était d'indiquer des coordonnées GPS (une position cible) au robot (soit manuellement, soit en brut dans le code) afin qu'il y aille de manière autonome tout en évitant les obstacles.

Conditions des essais

Pour faire ces essais, il a d'abord fallu récupérer les coordonnées GPS de la position cible et les coordonnées du robot.

Pour récupérer les coordonnées du robot, un topic ROS2 appelé /fix est déjà publié dans le contenu de base du robot. Ce topic contient toutes les informations relatives au GPS du robot, incluant ses coordonnées.

Pour récupérer les coordonnées de la position cible, le robot y a été déplacé et les coordonnées du robot à cette position ont été relevées (grâce encore une fois au topic /fix). Les coordonnées ainsi récupérées étaient normalement les coordonnées GPS de la position cible.

Ces essais ont été réalisés dans l'enceinte de Polytech, à l'extérieur, d'abord entre les bâtiments B et D, puis vers le terrain de sport, et enfin proche de la Bibliothèque Universitaire. Il est déconseillé de réessayer entre les bâtiments B et D, puisqu'il y a des perturbations magnétiques qui affectent le magnétomètre de l'IMU du robot, ce qui empêche le robot de bien estimer sa position.

Essais réalisés

1. Le GPS seul

Le premier essai réalisé a été en utilisant uniquement les coordonnées du GPS. La distance entre le robot et la cible est calculée avec leur coordonnées respectives, puis le robot est supposé se diriger vers la cible, et s'arrêter quand il est à environ 2 mètres de la cible (précision maximale pour ce genre d'application).

En pratique, le robot se dirige vers la cible, mais dans la mauvaise direction. Ce

comportement est normal, la position du robot n'est pas estimée, donc le robot ne "sait pas" où il commence à se déplacer et comment il est orienté par rapport à la cible, ce qui entraîne ce problème de direction.

2. GPS + Odométrie

Pour résoudre ce problème d'estimation de position du robot, l'odométrie du robot a été ajoutée aux coordonnées GPS. Elle est utilisée pour estimer l'orientation, notamment l'angle yaw (angle suivant l'axe Z) du robot.

En pratique, le comportement du robot est sensiblement le même, ce qui est normal puisqu'estimer l'orientation du robot avec l'odométrie seule n'est pas précis ou fiable. Il faut l'associer à l'IMU du robot et faire un filtre pour avoir une bonne estimation.

3. GPS + filtre de Kalman

Après des recherches sur l'utilisation de filtres avec un GPS sur ROS2, il a été décidé d'utiliser 2 filtres de Kalman en combinaison avec la node navsat_transform du paquet robot_localization de ROS2 (présent de base sur les robots normalement). Cette méthode est la méthode classique utilisée dans ce genre de situation (validée ensuite par les ingénieurs de Génération Robot, entreprise ayant vendu les robots à Polytech).

Le 1er filtre de Kalman fusionne les données de l'odométrie et de l'IMU (combinées elles aussi à l'aide d'un script) du robot. La node navsat_transform permet de convertir les données brute du GPS en une odométrie métrique publiée ensuite dans un topic. Le 2nd filtre de Kalman permet de fusionner les données du 1er filtre et de navsat_transform.

Cette méthode était supposée renvoyer une estimation de position correcte du robot, calculer la distance entre le robot et la cible et corriger la trajectoire du robot afin qu'il atteigne la cible.

Cependant, en pratique, le robot tournait en boucle sur lui-même. De plus, une erreur indiquant que les données du topic /fix ne pouvait pas être converti était retourné.

Cette erreur était due au manque de transformation géométrique entre le repère de /fix et le repère de base des autres capteurs. En effet, /fix publie ses données dans une base appelé gps, mais cette base n'a aucune transformation permettant d'utiliser ses données. Une base relativement similaire a été trouvée, gps_antenna_link, ayant toutes les transformations nécessaires pour les essais. Mais, pour l'utiliser, il faudrait modifier des fichiers constructeurs. Il a donc été décidé de contacter Génération Robot avant de continuer.

Après discussion avec Génération Robot, il s'est avéré que l'IMU du robot était inversée par rapport à la norme ROS2, ce qui fausse l'estimation de position. Pour résoudre ce problème, il faut aller dans le fichier gr_p247.urdf.xacro du paquet gr_description et regarder la partie suivante :

```
<xacro:include filename="$(find gr_description)/urdf/accessories/phidget_spatial.urdf.xacro" />
<xacro:phidget_spatial prefix="$(arg prefix)" parent="$(arg prefix)bottom_box_link" >
  <origin xyz="0.0 0.0 0.050" rpy="{pi} 0 0"/>
</xacro:phidget_spatial>
```

Si rpy= "0 0 0", il faut la modifier en "\${pi} 0 0" et cela devrait résoudre le problème de l'IMU.

Pour le problème du manque de transformation, Génération Robot nous a conseillé d'éviter de modifier les paquets constructeurs et de "surcharger la configuration" en faisant comme ceci :

- Copier/coller le contenu du launcher du constructeur situé ici : [ros2-ublox-zedf9p/ublox_gps/launch/ublox_gps_node_zedf9p-launch.py](#) vers [gr_bringup/launch/component_gps.launch.py](#)
- Copier/coller la configuration par défaut du constructeur [ros2-ublox-zedf9p/ublox_gps/config/zed_f9p.yaml](#) vers [gr_bringup/config/zed_f9p.yaml](#)
- Modifier la `frame_id` en `gps_antenna_link` dans [gr_bringup/config/zed_f9p.yaml](#)
- Modifier le chemin `config_directory` dans [gr_bringup/launch/component_gps.launch.py](#) afin de pointer vers votre fichier de configuration, comme ceci :

```
config_directory = os.path.join(
    ament_index_python.packages.get_package_share_directory('gr_bringup'),
    'config')
```

Il faut ensuite recompiler le code et relancer le robot, normalement, la base sur laquelle /fix publie les données devraient être devenue `gps_antenna_link`.

Si cela n'est pas le cas, modifiez le fichier [ros2-ublox-zedf9p/config/zed_f9p.yaml](#) pour mettre `gps_antenna_link` dans la partie du gps, recompilez et redémarrez le robot. Le changement devrait être fait, mais c'est une modification d'un fichier/paquet constructeur, ce qui peut peut-être créer des problèmes.

Après ces modifications, le robot a arrêté de tourner en permanence sur lui-même. Mais, d'autres problèmes sont apparus :

- un décalage suivant l'axe Z, indiquant un problème sur l'angle yaw;
- le calcul de distance est parfois incorrect au lancement du script et cette erreur est conservé sur toute la durée de l'exécution;
- même quand le calcul de distance est correct, à la 6e seconde de l'essai, la valeur de distance triple puis augmente très rapidement.

Pour résoudre le problème de l'angle yaw, nous avons essayé d'ajouter un filtre au script renvoyant les données fusionnées de l'IMU. Pour ce faire, nous avons installé le paquet *ros-humble-imu-filter-madgwick* et modifié le fichier launch lançant toutes les nodes.

Cette solution a réduit le décalage, mais il en restait toujours un léger. Pour le résoudre, nous avons ajouté un décalage dans `navsat_transform` égal au décalage réel. Mais le problème persiste.

Pour résoudre l'erreur de distance, nous avons essayé 2 méthodes :

- passer par un service (from_LL) pour convertir les coordonnées du GPS en coordonnées cartésiennes pour simplifier le calcul;
- utiliser la formule d'Haversine pour prendre en compte la courbure de la Terre et calculer la distance avec les valeurs brutes de /fix.

Ces 2 méthodes ont leurs avantages et inconvénients, mais dans les 2 cas, le problème de distance n'a pas été réglé.

Pour le problème de "saut" de distance (dérive temporelle), nous avons supposé que cela était peut être lié à la précision du GPS et le fait que nous utilisions un GPS standard, et non un GPS associé à un header.

De plus, un autre problème a été observé : aléatoirement, les données de /fix ne se mettent plus à jour, rendant donc impossible la navigation correcte du robot. Pour résoudre ce problème, il suffit de redémarrer le robot, aucune solution liée au software a été trouvée actuellement.

4. GPS et RTK

Une dernière méthode a été essayée pour l'utilisation du GPS. La dernière supposition que nous avons sur le comportement du robot était que la précision de nos méthodes ou du module GPS était trop faible. Pour ce faire, nous pouvions associer le GPS à un header, ici le RTK, pour augmenter cette précision. Le RTK consiste à se connecter à une antenne afin de recevoir des données plus précises. Pour se connecter, nous avons choisis d'utiliser le réseau NTRIP, c'est à dire se connecter via internet à une antenne proche. Nous avons aussi choisi de nous connecter au réseau centipedeRTK pour les essais avec la commande suivante :

```
sudo apt install ros-humble-ntrip-client
ros2 launch ntrip_client ntrip_client_launch.py \
  host:="crtk.net" \
  port:=443 \
  ssl:="true" \
  mountpoint:="SOPH" \
  authenticate:="true" \
  username:="centipede" \
  password:="centipede"
```

La commande *sudo* est à faire si le paquet n'est pas déjà installée sur le robot.

Avec la connection à l'antenne de Sophia, nous recevons bien des données plus précises, mais il était impossible de modifier l'état du GPS pour exploiter ces dernières sans modifier plusieurs fichiers constructeurs (au moins une dizaine). La méthode du GPS et du RTK a donc été abandonnée.

Etat actuel

Actuellement, tous les scripts utilisés pour les différents essais sont disponibles sur le dépôt Github du projet.

De plus, lors de l'exécution, le robot essaie bien de rejoindre la position cible, son comportement et sa trajectoire prise sont juste aléatoires.

Les problèmes à résoudre sont les suivants :

- le décalage/la dérive de l'angle yaw;
- le calcul de distance;
- les "sauts" de distance notamment à la 6eme seconde de chaque exécution (normalement résolvable avec le RTK).

Nous n'avons malheureusement pas d'autres pistes que de modifier les fichiers constructeurs du robot pour faire fonctionner le RTK, ou alors recontacter Génération Robot.

A noter que ces essais ont été effectués sur un seul robot, peut-être que les faire sur d'autres apporterait des résultats différents.

Une autre remarque est que, peut-être que les robots n'ont pas été conçus pour ce type d'application, étant donné le nombre de fichiers ou paquets constructeurs à modifier.